

Cyber Campo de Treinamento Educacional: Instrumentação, Detecção e Defesa Ativa para Ensino de Cibersegurança

Vinícius Dorneles Souza Ferreira¹, Josiane Fontoura dos Anjos

¹Instituto Federal Farroupilha – Campus Alegrete (IFFar)
RS-377 - Km 27 - Passo Novo - CEP 97555-000 - Alegrete/RS – Brasil

4lnicius.aleg@gmail.com, josiane.fontoura@iffar.edu.br

Resumo. *Este trabalho apresenta o desenvolvimento do Cyber Campo de Treinamento Educacional (CCTE), uma plataforma web educacional de cibersegurança no formato de cyber range, destinada ao aprendizado prático de segurança de aplicações em contexto acadêmico. O sistema disponibiliza um ambiente web propositalmente vulnerável e executado de forma isolada, permitindo a simulação ética e controlada de classes comuns de vulnerabilidades, como SQL Injection, Cross-Site Scripting (XSS), Insecure Direct Object Reference (IDOR), Path Traversal, File Disclosure e Brute Force. O desenvolvimento seguirá uma abordagem iterativa e incremental, organizada em seis etapas. As interações serão instrumentadas por um módulo de monitoramento e detecção, responsável por registrar eventos, identificar padrões suspeitos e gerar logs para análise técnica e forense. Como resultados esperados, pretende-se entregar uma ferramenta acessível ao contexto acadêmico, capaz de apoiar o ensino experiencial ao expor o ciclo completo de um incidente de segurança — tentativa, detecção, registro, análise e resposta — validada por meio de questionário aplicado a estudantes, docentes e profissionais da área, acompanhada de documentação técnica e materiais de apoio.*

Palavras-chave: *cibersegurança; cyber range; segurança web; honeypot; detecção de intrusão; análise forense.*

1. Introdução

Com o avanço das tecnologias digitais e a crescente dependência de sistemas computacionais em atividades acadêmicas, empresariais e governamentais, a cibersegurança tornou-se um elemento essencial para a proteção de dados, serviços e infraestruturas digitais. Aplicações web passaram a ocupar papel central nesse cenário devido à sua ampla utilização e acessibilidade, tornando-se também alvos frequentes de ataques cibernéticos. Segundo [OWASP Foundation 2021c], vulnerabilidades relacionadas a falhas de autenticação, injeções de código, exposição de informações sensíveis e problemas de controle de acesso figuram entre os riscos mais recorrentes em aplicações modernas.

A proteção desses sistemas tornou-se ainda mais relevante diante do aumento na quantidade e na sofisticação dos ataques digitais, impulsionados pela expansão da conectividade e pela crescente integração entre sistemas e serviços online. Além do impacto técnico, ameaças cibernéticas também passaram a representar riscos estratégicos relacionados à soberania digital, à proteção

de infraestruturas críticas e à segurança de organizações públicas e privadas [NIST — National Institute of Standards and Technology 2024].

Apesar da relevância do tema, o ensino de cibersegurança ainda enfrenta limitações significativas no contexto educacional. Muitos estudantes possuem contato predominantemente teórico com conceitos de segurança, mas encontram dificuldades em compreender, na prática, como vulnerabilidades são exploradas, monitoradas e mitigadas em ambientes reais.

Ambientes controlados e propositalmente vulneráveis possibilitam ao estudante observar o comportamento de ataques, compreender falhas de implementação e analisar mecanismos de monitoramento e resposta de forma ética e segura. Nesse contexto, o desenvolvimento do presente trabalho permite a simulação controlada de vulnerabilidades comuns em aplicações web, incluindo SQL Injection, Cross-Site Scripting (XSS), Insecure Direct Object Reference (IDOR), ataques de força bruta e Path Traversal/File Disclosure. Além da simulação de ataques, o ambiente integra mecanismos de monitoramento capazes de registrar eventos, identificar padrões suspeitos e gerar logs voltados à auditoria e análise forense.

Como resultado, espera-se disponibilizar uma ferramenta acessível capaz de apoiar o ensino experiencial em cibersegurança ao permitir que estudantes observem o ciclo completo de um incidente, envolvendo tentativa de exploração, detecção, registro, análise e resposta, em um ambiente seguro, controlado e eticamente isolado.

2. Objetivo Geral

A partir das limitações identificadas no ensino de cibersegurança e da ausência de ferramentas que integrem simulação, monitoramento e defesa ativa em um único ambiente, o presente trabalho tem como objetivo geral desenvolver e avaliar um cyber range web educacional com páginas vulneráveis instrumentadas, integrando monitoramento, detecção e mecanismos de defesa ativa para treinamento prático em cibersegurança.

2.1. Objetivos Específicos

Para cumprir o objetivo geral, o CCTE visa gerar os seguintes benefícios ao estudante e à comunidade acadêmica:

1. Proporcionar ao estudante um ambiente prático e seguro para a compreensão de vulnerabilidades web recorrentes, como SQL Injection, XSS, IDOR, Path Traversal, File Disclosure e Brute Force, sem necessidade de infraestrutura complexa;
2. Permitir que o estudante observe o ciclo completo de um incidente de segurança, envolvendo tentativa de exploração, detecção, registro, análise e resposta, integrando as perspectivas ofensiva e defensiva em um único ambiente;
3. Oferecer à comunidade acadêmica uma ferramenta educacional de baixo custo operacional, executável localmente sem dependência de serviços externos ou contêineres, viabilizando sua adoção em instituições com infraestrutura limitada;
4. Disponibilizar mecanismos de monitoramento e geração de logs que permitam ao estudante desenvolver habilidades de análise forense digital, identificando padrões de ataque e reconstruindo a linha do tempo de incidentes;
5. Contribuir para a formação de profissionais mais qualificados em segurança da

informação, aproximando o aprendizado acadêmico de situações reais encontradas em ambientes de produção;

6. Fornecer à comunidade docente uma plataforma que apoie o ensino experiencial de cibersegurança, com laboratórios organizados por tipo de vulnerabilidade e recursos de avaliação integrados.

3. Cibersegurança

A cibersegurança compreende o conjunto de práticas, tecnologias e processos destinados à proteção de sistemas computacionais, redes, aplicações e dados contra acessos não autorizados, ataques e danos intencionais ou acidentais. Seu principal objetivo é garantir a segurança das informações e a continuidade dos serviços digitais, preservando a confiabilidade dos sistemas utilizados por indivíduos, organizações e governos [NIST — National Institute of Standards and Technology 2024].

Com a crescente digitalização da sociedade, atividades como comunicação, serviços financeiros, armazenamento de dados e operações governamentais passaram a depender diretamente de infraestruturas digitais, tornando a cibersegurança um elemento estratégico. Nesse contexto, a proteção de sistemas tornou-se não apenas uma necessidade técnica, mas também um fator relacionado à estabilidade econômica, à soberania digital e à segurança nacional [ENISA — European Union Agency for Cybersecurity 2024].

Conforme [IBM 2024b], um dos fundamentos centrais da segurança da informação é a tríade CIA (Confidentiality, Integrity and Availability), composta pelos princípios de confidencialidade, integridade e disponibilidade. A confidencialidade garante que informações sejam acessadas apenas por entidades autorizadas; a integridade assegura que os dados não sejam alterados indevidamente; e a disponibilidade garante que sistemas e informações permaneçam acessíveis quando necessários.

Paralelamente ao avanço tecnológico, observa-se o crescimento da quantidade e da sofisticação dos ataques digitais. Aplicações web, devido à sua ampla exposição e integração com serviços críticos, tornaram-se alvos frequentes de exploração de vulnerabilidades, vazamentos de dados e ataques automatizados. Esse cenário evidencia a necessidade de profissionais qualificados capazes de compreender, detectar e mitigar ameaças em ambientes computacionais [OWASP Foundation 2021c].

Dessa forma, o ensino de cibersegurança demanda não apenas fundamentação teórica, mas também ambientes práticos que permitam a simulação segura de cenários de ataque e defesa, aproximando o aprendizado acadêmico de situações reais encontradas em sistemas computacionais.

3.1. Honeypot

Conforme [IBM 2024a], o honeypot é um mecanismo de segurança projetado para simular vulnerabilidades de forma controlada, com o objetivo de atrair, registrar e analisar interações potencialmente maliciosas, priorizando a observação e coleta de informações relacionadas ao comportamento do atacante em detrimento de mecanismos tradicionais de proteção.

A principal característica do honeypot implementado é a detecção passiva, na qual

o sistema registra interações sem interferir diretamente no comportamento observado. Os eventos são armazenados em logs de auditoria e logs de segurança, contendo informações relacionadas a autenticação, navegação, tentativas de exploração e comportamentos suspeitos.

Esses registros permitem a realização de análise forense digital, possibilitando reconstrução de eventos, identificação de padrões e estudo das técnicas utilizadas durante as interações. Dessa forma, o honeypot educacional atua como ferramenta de aprendizado prático voltada à observação do ciclo de exploração e monitoramento em aplicações web [Kaspersky 2023].

3.2. Cyber Range

Segundo [NIST — National Institute of Standards and Technology 2018], o Cyber Range consiste em um ambiente controlado destinado ao treinamento prático em cibersegurança, permitindo a simulação segura de ataques, vulnerabilidades e mecanismos de defesa, sendo utilizado por instituições acadêmicas, organizações privadas e entidades governamentais para capacitação técnica e treinamento operacional.

O Cyber Range possibilita interação prática com sistemas vulneráveis e cenários simulados, favorecendo o aprendizado experiencial em segurança ofensiva e defensiva, aproximando o estudante de situações reais encontradas em ambientes de produção. A integração desses componentes permite que o usuário observe o ciclo completo de um incidente de segurança, envolvendo tentativa de exploração, detecção, registro, análise e resposta, proporcionando uma visão abrangente tanto da perspectiva ofensiva quanto da defensiva. Além disso, o ambiente opera localmente e com dados simulados, garantindo que as atividades sejam realizadas de forma ética, controlada e sem riscos a sistemas reais, o que viabiliza sua adoção em contextos acadêmicos com infraestrutura limitada.

4. Vulnerabilidades em aplicações web

Vulnerabilidades em aplicações web correspondem a falhas presentes no desenvolvimento, configuração ou implementação de sistemas acessíveis via navegador, que podem ser exploradas para comprometer dados, usuários ou funcionalidades da aplicação. Essas falhas geralmente decorrem de validação inadequada de entradas, erros de controle de acesso, gerenciamento incorreto de sessões ou implementação insegura de mecanismos de autenticação [OWASP Foundation 2021c].

Devido à ampla exposição das aplicações web e sua integração com bancos de dados e serviços críticos, esse tipo de sistema tornou-se um dos principais alvos de ataques cibernéticos. Organizações como a OWASP documentam e classificam vulnerabilidades recorrentes em aplicações modernas, auxiliando no desenvolvimento de práticas mais seguras. As subseções a seguir detalham cada uma das vulnerabilidades implementadas no CCTE, bem como os conceitos de honeypot e cyber range que fundamentam sua arquitetura.

4.1. SQL Injection

Segundo [OWASP Foundation 2021e], SQL Injection ocorre quando entradas fornecidas pelo usuário são inseridas diretamente em consultas SQL sem validação ou separação

adequada entre dados e comandos, permitindo que um atacante manipule a lógica da consulta executada pelo banco de dados.

Essa vulnerabilidade surge principalmente da concatenação direta de entradas do usuário em instruções SQL, permitindo que os dados sejam interpretados como comandos executáveis. Como consequência, um atacante pode acessar informações sensíveis, contornar mecanismos de autenticação, modificar registros ou comprometer completamente o banco de dados.

4.2. Cross-Site Scripting

De acordo com [OWASP Foundation 2021a], o Cross-Site Scripting (XSS) é uma vulnerabilidade que permite a inserção de scripts maliciosos em páginas web visualizadas por outros usuários, ocorrendo quando a aplicação não realiza validação ou sanitização adequada dos dados antes de exibi-los na interface.

Na prática, o atacante injeta código, geralmente em JavaScript, que será executado no navegador da vítima dentro do contexto da aplicação comprometida. Os impactos incluem roubo de sessões, manipulação de interface e execução de ações em nome do usuário afetado.

4.3. Insecure Direct Object Reference

A vulnerabilidade de IDOR (Insecure Direct Object Reference) ocorre quando a aplicação expõe identificadores internos, como IDs de usuários ou registros, sem verificar adequadamente as permissões de acesso [OWASP Foundation 2021b].

Nesse cenário, usuários autenticados podem manipular parâmetros em URLs ou requisições para acessar dados pertencentes a outros usuários, caracterizando uma falha de controle de acesso. Os impactos incluem violação de privacidade, exposição de informações sensíveis e quebra do isolamento entre contas.

4.4. Path Traversal

O Path Traversal é uma vulnerabilidade que permite ao usuário manipular caminhos de arquivos para acessar diretórios fora do escopo previsto pela aplicação. Essa falha ocorre quando parâmetros recebidos pelo sistema são utilizados diretamente para localizar arquivos no servidor sem validação adequada [OWASP Foundation 2021d].

A exploração dessa vulnerabilidade pode permitir acesso a arquivos internos, códigos-fonte, configurações e outros recursos restritos do sistema, comprometendo a segurança da aplicação e facilitando ataques subsequentes.

4.5. File Disclosure

O File Disclosure refere-se à exposição indevida de arquivos sensíveis armazenados no servidor e acessíveis ao usuário sem autorização adequada. Diferentemente do Path Traversal, essa vulnerabilidade não depende necessariamente da manipulação de caminhos, mas da falha em restringir corretamente o acesso a recursos internos [PortSwigger 2024].

Arquivos contendo credenciais, logs, backups ou bancos de dados podem ser expostos por falhas de configuração ou ausência de controles de acesso, resultando em va-

zamento de informações críticas e comprometimento da aplicação.

4.6. Brute Force

O ataque de força bruta consiste na realização repetitiva de tentativas de autenticação até que uma combinação válida de credenciais seja encontrada. Embora não seja uma vulnerabilidade diretamente relacionada ao código da aplicação, sua eficácia está associada à ausência de mecanismos de proteção, como limitação de tentativas e políticas adequadas de senha [Cloudflare 2024].

Sistemas que utilizam credenciais previsíveis ou não implementam mecanismos de mitigação tornam-se mais suscetíveis a esse tipo de ataque.

5. Trabalhos Relacionados

A fim de contextualizar o CCTE no panorama atual de ferramentas educacionais em cibersegurança, esta seção apresenta plataformas e estudos relacionados, organizados em três categorias: ambientes web propositalmente vulneráveis, honeypots com fins educacionais e cyber ranges no ensino de cibersegurança. A análise dessas referências evidencia lacunas que o CCTE se propõe a preencher.

5.1. Ambientes Web propositalmente vulneráveis

A Damn Vulnerable Web Application (DVWA) é uma aplicação PHP/MySQL com falhas de segurança distribuídas em três níveis de dificuldade (baixo, médio e alto), voltada principalmente para iniciantes que desejam compreender as vulnerabilidades do OWASP Top 10. Cada categoria de vulnerabilidade possui uma página com visualização do código-fonte vulnerável, facilitando a compreensão da falha de implementação. Sua principal limitação está na interface, que não simula uma aplicação real, exigindo que o estudante realize uma transposição mental entre o ambiente de laboratório e cenários do mundo real [Wood 2023].

O OWASP Juice Shop é considerado a aplicação vulnerável mais sofisticada disponível atualmente. Desenvolvida em Node.js e Angular, simula uma loja de comércio eletrônico com mais de cem desafios organizados por dificuldade, um painel de pontuação integrado e design gamificado no estilo CTF (Capture The Flag). Sua interface realista a torna especialmente adequada para treinamento de desenvolvedores que trabalham com tecnologias JavaScript e APIs REST modernas[OWASP Foundation 2023].

5.2. Honeypots com Fins Educacionais

O uso de honeypots como ferramenta de ensino em segurança da informação é abordado em diferentes trabalhos da literatura. Em estudo publicado no ResearchGate, pesquisadores defendem o honeypot como instrumento educacional eficaz para o estudo de questões de segurança em redes, demonstrando sua aplicação em projetos acadêmicos distribuídos com registro e análise de comportamento de atacantes. O trabalho destaca que honeypots fornecem pequenas quantidades de dados altamente relevantes, facilitando a compreensão por estudantes e a condução de pesquisas futuras [Negi et al. 2020].

Sikdar, Basu e Kule (2024), em trabalho apresentado no COMSYS 2023 e publi-

cado pela Springer, propõem um modelo de honeypot em três níveis hierárquicos para detecção e prevenção de intrusões web. O sistema detecta ataques na primeira camada, identifica modificações realizadas pelo atacante na segunda e, na terceira, direciona o agente para um banco de dados ou diretório falso enquanto coleta informações detalhadas. A abordagem reforça a viabilidade de honeypots instrumentados como mecanismo de observação e resposta em ambientes web [Sikdar et al. 2024].

5.3. Cyber Ranges no Ensino de Cibersegurança

Saddiqa, Broholm e Pedersen (2024), em trabalho apresentado na Cyber Science 2023 (Springer), investigam como projetar plataformas de treinamento em cibersegurança acessíveis a professores de ensino médio e técnico. O estudo identifica que a usabilidade é um fator crítico para a adoção dessas ferramentas no contexto educacional, destacando a necessidade de ambientes com exercícios organizados por nível de conhecimento e de fácil acesso por parte dos docentes [Saddiqa et al. 2024].

5.4. Comparativo

Em relação ao foco, DVWA e OWASP Juice Shop concentram-se na exploração ofensiva de vulnerabilidades web, em PHP e JavaScript, respectivamente, sem abordar a perspectiva defensiva. O CCTE proporá o ciclo completo de um incidente, integrando simulação, detecção, registro e resposta.

Quanto ao monitoramento e ao honeypot, nenhuma das plataformas consolidadas oferece esses recursos nativamente. O CCTE implementará dois pipelines de log distintos (audit.log e security.log) e um modo Honeypot Educacional de coleta passiva de interações, viabilizando análise forense posterior.

Por fim, DVWA e Juice Shop dependem de Docker para execução, o que pode representar uma barreira em ambientes com infraestrutura limitada. O CCTE operará diretamente sobre Python, Flask e SQLite, sem necessidade de contêineres.

Tabela 1. Comparativo entre plataformas de treinamento em cibersegurança.

Plataforma	Foco	Monitoramento	Honeypot	Execução
DVWA	Vulnerabilidade web em PHP	–	–	Docker/LAMP
OWASP JS	Vulnerabilidade web em JavaScript	–	–	Docker/Node.js
CCTE	Ciclo completo	Logs de auditoria e segurança	Educacional	Python/Flask/SQLite

Diante desse comparativo, o CCTE se posiciona como uma alternativa complementar às plataformas existentes, diferenciando-se pela integração do ciclo defensivo ao ambiente de treinamento e pela ausência de dependências de infraestrutura complexa.

6. Metodologia

Esta seção descreve os procedimentos metodológicos adotados no desenvolvimento do CCTE, abrangendo o tipo de pesquisa, as etapas de desenvolvimento do sistema, as tecno-

logias utilizadas e o planejamento da avaliação do protótipo. A abordagem metodológica será estruturada de forma a garantir coerência entre os objetivos propostos e as decisões técnicas tomadas ao longo do processo de construção da plataforma.

6.1. Tipo de pesquisa

Este trabalho caracteriza-se como uma pesquisa aplicada. Conforme [Gil 2002], pesquisas aplicadas buscam gerar conhecimento direcionado à resolução de problemas específicos e à aplicação prática dos resultados obtidos, caracterização que se aplica ao presente trabalho, cujo objetivo é o desenvolvimento de um artefato tecnológico voltado ao ensino de cibersegurança.

Quanto aos objetivos, a pesquisa possuirá caráter exploratório e descritivo. Exploratório por investigar conceitos, ferramentas e abordagens relacionadas a cyber ranges, vulnerabilidades web e mecanismos de monitoramento em ambientes educacionais; e descritivo por documentar a arquitetura, o funcionamento e as funcionalidades implementadas no sistema.

Em relação à abordagem metodológica, trata-se de uma pesquisa qualitativa, pois buscará analisar a percepção dos usuários sobre o potencial educacional da ferramenta, considerando aspectos relacionados à experiência prática, compreensão de vulnerabilidades e interpretação dos mecanismos de monitoramento e defesa.

6.2. Descrição do desenvolvimento do sistema

O desenvolvimento do CCTE seguirá uma abordagem iterativa e incremental, permitindo a implementação gradual das funcionalidades e validações parciais ao longo do processo.

1. Levantamento de requisitos e definição do escopo: serão identificados os principais tipos de vulnerabilidades a serem simuladas, os modos operacionais do sistema (Honeypot Educacional e Defesa Ativa) e os requisitos funcionais relacionados ao monitoramento, geração de logs e relatórios técnicos;
2. Modelagem da arquitetura: será definida a estrutura modular do sistema, incluindo a separação entre os módulos de páginas vulneráveis (labs), o pipeline de auditoria de eventos e o módulo de detecção de comportamentos suspeitos;
3. Implementação do ambiente vulnerável: serão desenvolvidas as páginas propositalmente vulneráveis, estruturadas em laboratórios independentes, cada um abordando um tipo específico de vulnerabilidade (SQL Injection, XSS, IDOR, Path Traversal, File Disclosure e Brute Force);
4. Implementação do sistema de monitoramento: será definido e implementado o modelo de eventos de segurança (log schema), bem como o pipeline de auditoria responsável pelo registro, classificação e armazenamento das interações realizadas no sistema;
5. Implementação dos modos operacionais: serão desenvolvidos os dois modos de operação do sistema, o Honeypot Educacional, focado na coleta passiva de dados, e a Defesa Ativa, com mecanismos de mitigação como rate limiting e bloqueio de acesso;
6. Geração de relatórios: será implementada a funcionalidade de geração de relatórios técnicos, contendo linha do tempo de eventos, origem das requisições, endpoints acessados e evidências associadas.

6.3. Tecnologias utilizadas

O sistema será desenvolvido em Python 3.13, utilizando o framework Flask para o backend, escolhido por sua simplicidade e pelo controle direto sobre o comportamento da aplicação, característica essencial para a implementação proposital de vulnerabilidades. Para persistência de dados, será utilizado o SQLite, cuja integração nativa com o ecossistema Python dispensa soluções mais complexas em ambiente local. A interface será desenvolvida em HTML e CSS com templates Jinja2, e o desenvolvimento será realizado na IDE PyCharm. Todo o ambiente operará localmente, sem dependência de serviços externos, garantindo que as simulações ocorram de forma ética, segura e controlada.

6.4. Avaliação do Sistema

A avaliação do protótipo terá como objetivo analisar a efetividade do sistema como ferramenta educacional de apoio ao ensino de cibersegurança. A participação será voluntária e os dados coletados serão utilizados exclusivamente para fins acadêmicos, preservando o anonimato dos participantes.

- **Participantes:** a avaliação será realizada com participantes de diferentes níveis de experiência, incluindo estudantes de ensino técnico, estudantes de graduação, docentes, profissionais da área de TI e entusiastas em segurança da informação;
- **Instrumento de Coleta:** a coleta de dados será realizada por meio de questionário online aplicado após a utilização do sistema, estruturado em três blocos: perfil do participante, avaliação de usabilidade e percepção de aprendizado;
- **Dimensões Avaliadas:** o questionário buscará identificar a facilidade de uso do sistema, a compreensão das vulnerabilidades apresentadas, o entendimento do ciclo de incidente e a percepção do valor educacional da ferramenta;
- **Análise dos Dados:** os dados quantitativos serão analisados por meio de estatística descritiva, utilizando métricas como frequência, média e distribuição das respostas. As respostas abertas serão analisadas qualitativamente por categorização temática, buscando identificar dificuldades encontradas, percepção de aprendizado, sugestões de melhoria e padrões de utilização do sistema.

6.5. Arquitetura, Diagramas e Modelagem de Sistema

Esta seção apresenta a arquitetura geral do CCTE, os diagramas de caso de uso e a modelagem do banco de dados, oferecendo uma visão estrutural do sistema a ser desenvolvido e das relações entre seus componentes, como ilustra a Figura 1.

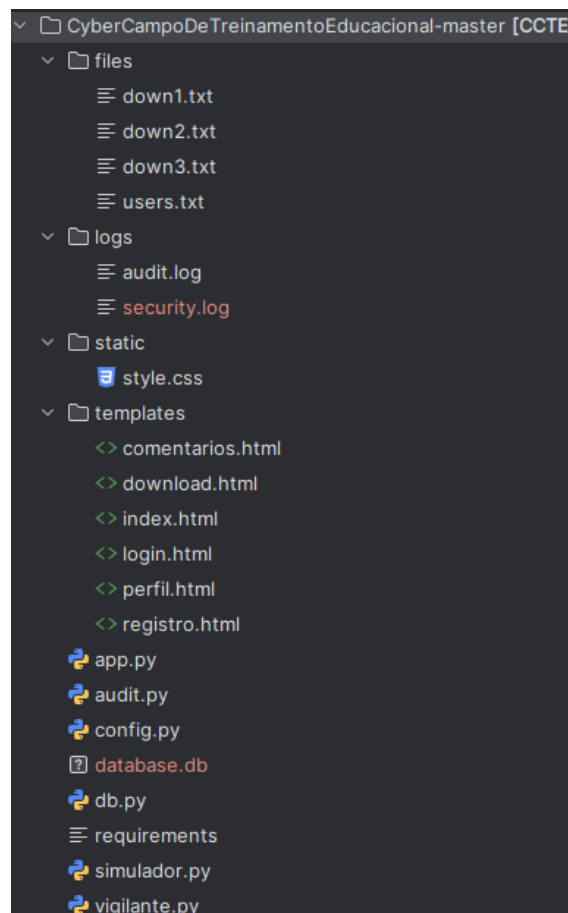


Figura 1. Arquitetura geral do sistema CCTE.

A Tabela 2 detalha os componentes que comporão a estrutura do sistema, descrevendo o tipo e a função de cada elemento presente na raiz do projeto.

6.5.1. Caso de Uso: Sistema Geral

O diagrama de caso de uso da Figura 2 apresentará as interações do único ator do sistema, o Usuário, com as funcionalidades disponíveis na plataforma CCTE, organizadas em quatro grupos funcionais.

Autenticação: compreenderá as ações de registrar conta, fazer login e fazer logout, garantindo o controle de acesso à plataforma.

Navegação: englobará as funções de navegar pelo menu, ver a página inicial e usar o botão de dica, que orienta o usuário durante os exercícios práticos.

Vulnerabilidades: núcleo didático do sistema, permitirá ao usuário praticar as classes de vulnerabilidades web implementadas: SQL Injection, XSS, IDOR, Path Traversal, File Disclosure e Brute Force.

Monitoramento: disponibilizará recursos de análise e resposta a incidentes, incluindo a visualização de logs de auditoria e de segurança, a geração de relatório técnico e a alternância do modo operacional do sistema.

Tabela 2. Componentes do sistema CCTE

Componente	Tipo	Descrição
CCTE/	Raiz	Pasta raiz do sistema
files/	Diretório	Arquivos disponíveis para download e simulação de Path Traversal
logs/	Diretório	Logs de auditoria e segurança gerados pelo sistema
static/	Diretório	Recursos estáticos do sistema com arquivos de estilo (CSS)
templates/	Diretório	Páginas HTML do sistema renderizadas via Jinja2
app.py	Módulo	Aplicação principal do servidor Flask e definição de rotas do sistema
audit.py	Módulo	Script de auditoria: geração de logs de eventos
vigilante.py	Módulo	Script de monitoramento: detecção de comportamentos suspeitos
db.py	Módulo	Script de banco de dados: inicialização e conexão
config.py	Módulo	Configurações globais do sistema
simulador.py	Módulo	Script de simulação automatizada de ataques
database.db	Arquivo	Banco de dados SQLite local

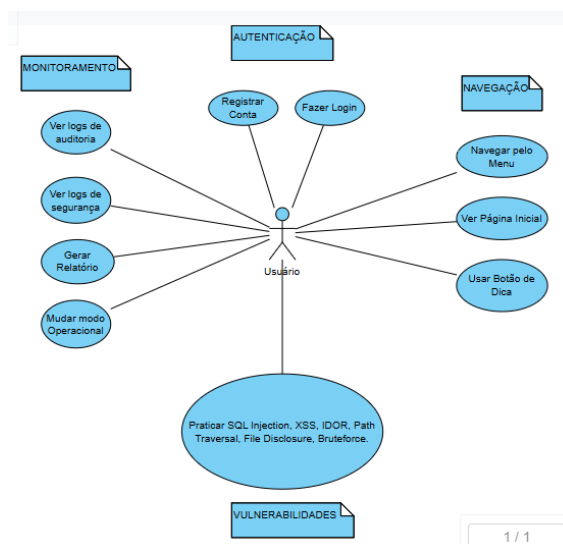


Figura 2. Diagrama de caso de uso — sistema geral do CCTE.

6.5.2. Caso de Uso: Modo Operacional

O diagrama da Figura 3 detalha o caso de uso Alternar Modo Operacional, funcionalidade do módulo de monitoramento do CCTE. A partir dessa ação, o sistema assumirá dois

modos distintos modelados como extensões (`<<extend>>`): o Modo Honeypot Forense, em que o sistema registrará interações de forma passiva para coleta de evidências, e o Modo Defesa Ativa, em que adotará postura reativa, podendo bloquear IPs suspeitos, aplicar rate limit, restringir acesso e deletar scripts maliciosos detectados no ambiente.

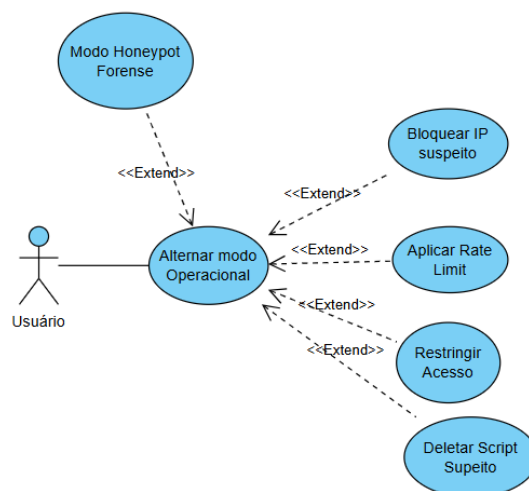


Figura 3. Diagrama de caso de uso — modo operacional do CCTE.

6.5.3. Modelo Relacional de Banco de Dados

O modelo da Figura 4 representará a estrutura relacional do CCTE, composta por duas entidades. A tabela `users` armazenará os dados de autenticação, contendo identificador, nome de usuário, senha e flag de administrador (`is_admin`). A tabela `comentarios` registrará as interações textuais da plataforma, armazenando o identificador, o usuário autor e o conteúdo. As entidades se relacionarão por meio da associação `publica`, com cardinalidade $(1, 1) \text{ -- } (0, n)$, indicando que um usuário poderá publicar nenhum ou múltiplos comentários, enquanto cada comentário pertencerá a exatamente um usuário.

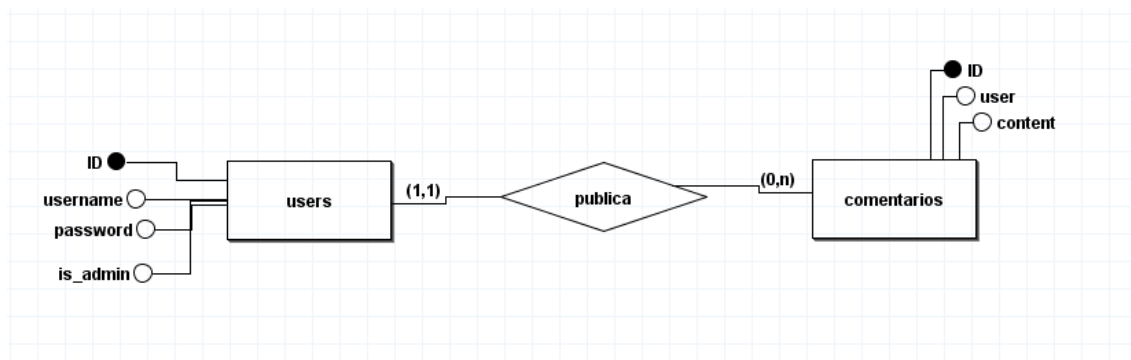


Figura 4. Modelo relacional de banco de dados do CCTE.

6.5.4. Modelo Lógico de Banco de Dados

O modelo lógico da Figura 5 detalhará os tipos de dados de cada atributo das entidades do CCTE. A tabela `users` definirá `ID` como chave primária do tipo `INTEGER`, além

dos campos `username` e `password` como `TEXT`. O campo `is_admin` será do tipo `INTEGER` e controlará o nível de privilégio do usuário por convenção numérica: o valor 0 identificará um administrador, enquanto valores diferentes de 0 — convencionalmente 1 — identificarão usuários comuns. A tabela `comentarios` possuirá `ID` como chave primária do tipo `INTEGER`, `user` como referência textual ao autor e `content` armazenando o texto da interação. A cardinalidade $(1, 1) \text{ -- } (0, n)$ indicará que cada comentário pertencerá a um único usuário, enquanto um usuário poderá ter nenhum ou múltiplos comentários registrados.

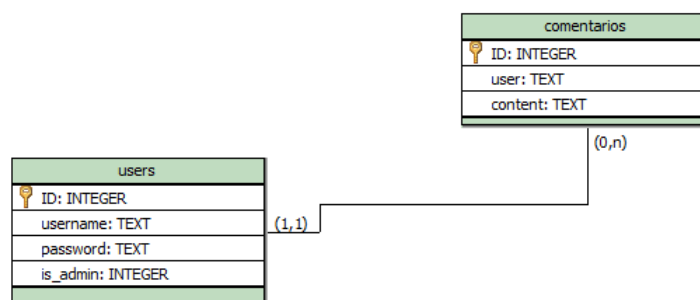


Figura 5. Modelo lógico de banco de dados do CCTE.

7. Desenvolvimento

Com base na arquitetura e na modelagem apresentadas na seção anterior, esta seção detalha os principais elementos de implementação do CCTE, incluindo o banco de dados, as páginas vulneráveis, o ambiente de execução e os pipelines de auditoria e detecção de eventos de segurança.

7.1. Banco de Dados

O CCTE utilizará SQLite como sistema de gerenciamento de banco de dados, solução escolhida por sua leveza e por dispensar a necessidade de um servidor dedicado, adequando-se ao contexto acadêmico da plataforma. O esquema será composto por duas tabelas criadas automaticamente na inicialização do sistema, caso ainda não existam.

A tabela `users` armazenará as credenciais de acesso, contendo os campos `id` (chave primária com autoincremento), `username`, `password` e `is_admin`, sendo este último um inteiro que distinguirá administradores (valor 0) de usuários comuns (valor 1), conforme ilustrado na Figura 6. A tabela `comentarios`, apresentada na Figura 7, registrará as interações da plataforma com os campos `id`, `user` e `content`.

Na inicialização, o sistema populará automaticamente a base com um conjunto de usuários padrão (`USUARIOS_PADRAO`), conforme a Figura 8, composto por quatro usuários comuns — *ana*, *carlos*, *joao* e *maria* — e dois administradores — *admin* e *sysadmin* — com credenciais intencionalmente fracas, característica proposital do ambiente vulnerável que viabilizará a prática de ataques como força bruta e injeção de SQL.

```

sql
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT,
  password TEXT,
  is_admin INTEGER
);

```

Figura 6. Criação da tabela users.

```

sql
CREATE TABLE IF NOT EXISTS comentarios (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user TEXT,
  content TEXT
);

```

Figura 7. Criação da tabela comentarios.

7.2. Páginas Vulneráveis

O sistema será composto por cinco rotas com vulnerabilidades educacionais implementadas intencionalmente, conforme a Tabela 3. As rotas /registro e /login exporão falhas de SQL Injection nos comandos INSERT e SELECT, respectivamente, sendo que /login também viabilizará ataques de Brute Force. A rota /perfil simulará IDOR, /comentarios exporá XSS e /download combinará Path Traversal e File Disclosure.

Tabela 3. Rotas vulneráveis e suas respectivas vulnerabilidades.

Rota	Template	Vulnerabilidade
/registro	registro.html	SQL Injection (INSERT)
/login	login.html	SQL Injection (SELECT) + Brute Force
/perfil	perfil.html	IDOR
/comentarios	comentarios.html	XSS
/download	download.html	Path Traversal + File Disclosure

7.3. Ambiente de Execução

O sistema operará em ambiente local isolado, executando um servidor Flask acessível via localhost. O ambiente será autocontido — banco de dados local, dados simulados e sem integrações externas — garantindo que todas as simulações ocorram sem exposição a redes externas ou riscos a sistemas reais.

7.4. Pipeline de Auditoria

O módulo de auditoria implementará um pipeline de registro de eventos de segurança baseado em arquivo de log estruturado. Cada evento será registrado com o seguinte esquema:

```

USUARIOS_PADRAO = [
    ("ana", "123456", 0),
    ("carlos", "qwerty", 0),
    ("joao", "admin123", 0),
    ("maria", "senha123", 0),
    ("admin", "admin", 1),
    ("sysadmin", "SuperSecure123", 1),
]

```

Figura 8. Usuários padrão inseridos na inicialização do sistema.

TIMESTAMP | IP | EVENTO | DETALHE

Tabela 4. Eventos registrados pelo módulo de auditoria.

Evento	Descrição
LOGIN_SUCESSO	Autenticação realizada com sucesso
FALHA_LOGIN	Credenciais inválidas
USUARIO_CRIADO	Novo usuário registrado no sistema
USUARIO_DUPLICADO	Username duplicado na tentativa de registro
PAGE_VIEW	Acesso a página monitorada
DOWNLOAD_REQUEST	Requisição de arquivo
DOWNLOAD_SUCCESS	Arquivo lido e retornado com sucesso
DOWNLOAD_FALHA	Arquivo não encontrado ou com erro de leitura
CAMPO_VAZIO	Campo de formulário obrigatório vazio
LOGOUT	Encerramento de sessão pelo usuário
COMENTARIO_POST	Publicação de comentário no sistema

Essa granularidade nos eventos permitirá, em análise posterior, a reconstrução completa da linha do tempo de um incidente, incluindo o momento exato, a origem e o comportamento do agente envolvido.

7.5. Pipeline do Log de Segurança

O módulo `vigilante.py` implementará um pipeline de detecção e registro de eventos suspeitos, separado do log de auditoria. Diferentemente do `audit.log`, que registrará todos os eventos relevantes do sistema, o `security.log` será escrito apenas quando pelo menos um tipo de ataque for detectado na requisição. Cada entrada seguirá o esquema:

TIMESTAMP | IP | MÉTODO ENDPOINT | TIPO(S) | PAYLOAD | ARGS
 | SESSION_USER

Tabela 5. Campos do log de segurança gerado pelo `vigilante.py`.

Campo	Origem	Descrição
TIMESTAMP	<code>horario()</code>	Data e hora do evento
IP	<code>ip</code>	Endereço IP de origem da requisição
ENDPOINT	<code>method + endpoint</code>	Método HTTP e rota acessada
TIPO(S)	<code>tipo(s)</code>	Categorias de ataque detectadas
PAYLOAD	<code>payload_str</code>	Conteúdo do corpo da requisição
ARGS	<code>args_str</code>	Parâmetros de URL
SESSION_USER	<code>session_user</code>	ID do usuário autenticado na sessão

8. Cronograma

O cronograma de desenvolvimento do CCTE, apresentado na Tabela 6, distribui as seis etapas do projeto ao longo de nove meses, de outubro de 2025 a junho de 2026. As etapas foram planejadas de forma incremental, com sobreposições pontuais que permitem a validação parcial das funcionalidades à medida que o desenvolvimento avança.

Tabela 6. Cronograma de desenvolvimento do CCTE.

Etapas	10/25	11/25	12/25	01/26	02/26	03/26	04/26	05/26	06/26
Levantamento de requisitos e definição do escopo	x	x	x						
Modelagem da arquitetura			x	x					
Implementação do ambiente vulnerável				x	x	x			
Implementação do sistema de monitoramento					x	x	x	x	
Implementação dos modos operacionais						x	x	x	
Geração de relatórios								x	x

9. Protótipo

A fim de ilustrar o funcionamento do CCTE e demonstrar a viabilidade técnica da proposta, esta seção apresenta um exemplo de uso da plataforma envolvendo a vulnerabilidade de SQL Injection, desde a interação do usuário até o registro nos logs de auditoria e segurança.

Como exemplo de uso, será possível cadastrar e autenticar um novo usuário e, em seguida, inserir um payload de SQL Injection nos campos de credenciais para acessar o perfil de outro usuário, demonstrando a vulnerabilidade de forma didática e controlada.

A Figura 9 apresentará a tela de login da plataforma, com o aviso de ambiente vulnerável no topo e o botão de dica no canto inferior direito. A Figura 10 exibirá a janela de dica acionada pelo usuário, descrevendo a vulnerabilidade disponível e sugerindo o payload `' OR 1=1 --` como exemplo didático. A Figura 11 mostrará a resposta do terminal, evidenciando a query manipulada e o alerta de SQL Injection detectado pelo módulo `vigilante.py`.

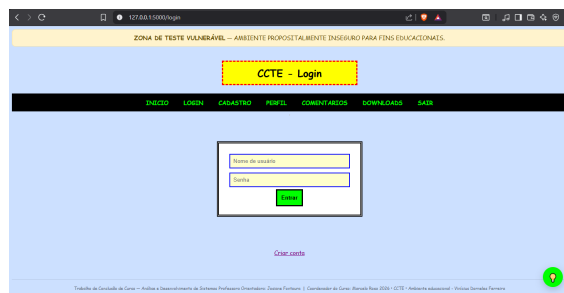


Figura 9. Tela de login do CCTE.

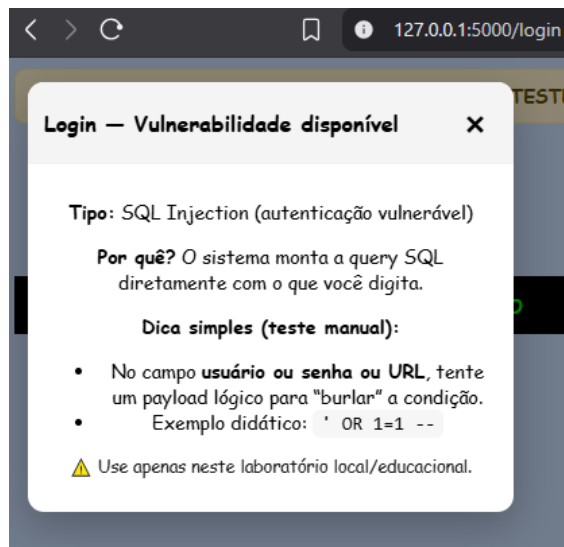


Figura 10. Janela de dica exibida ao usuário na rota /login.

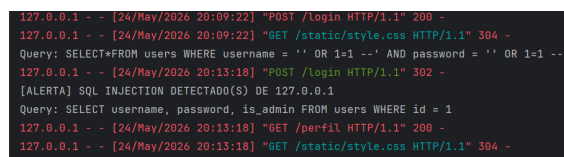


Figura 11. Resposta do terminal após execução do payload de SQL Injection.

Os registros gerados pelo ataque são apresentados a seguir. O log de segurança registrará a tentativa com o payload enviado:

```
2026-05-24 20:09:22 | 127.0.0.1 | POST /login | SQL INJECTION |
payload=ImmutableMultiDict([('username', "' OR 1=1 --"),
('password', '')]) | args= | session_user=None
```

O log forense registrará o acesso bem-sucedido ao perfil do usuário de ID 1 após a injeção:

```
2026-05-24 20:13:18 | 127.0.0.1 | LOGIN_SUCESSO | user_id=1
↪ username=ana
```

2026-05-24 20:13:18 | 127.0.0.1 | PAGE_VIEW |
↪ endpoint=/perfil
session_id=1 requested_id=None

10. Considerações

Este trabalho apresenta o desenvolvimento do Cyber Campo de Treinamento Educacional (CCTE), uma plataforma web educacional de cibersegurança projetada para apoiar o ensino prático de segurança de aplicações em contexto acadêmico, integrando simulação de vulnerabilidades, monitoramento de eventos e mecanismos de defesa ativa em um único ambiente local e isolado.

O protótipo a ser desenvolvido contemplará os principais objetivos definidos para o TCC1, com laboratórios para as vulnerabilidades SQL Injection, XSS, IDOR, Path Traversal, File Disclosure e Brute Force. O módulo de auditoria registrará eventos com granularidade suficiente para a reconstrução da linha do tempo de um incidente, enquanto o módulo de detecção identificará padrões suspeitos e os persistirá de forma separada no log de segurança, viabilizando análise forense posterior. Diferentemente de plataformas consolidadas como DVWA e OWASP Juice Shop, que priorizam a exploração ofensiva, o CCTE exporá ao estudante o ciclo completo de um incidente, incluindo a perspectiva defensiva de detecção, registro e resposta.

Os resultados obtidos até o momento indicam a viabilidade técnica da proposta. Como continuidade, o TCC2 prevê a avaliação do protótipo com o público-alvo por meio de questionário aplicado a estudantes, docentes e profissionais da área, com o objetivo de validar a efetividade da ferramenta e identificar oportunidades de melhoria na usabilidade e no conteúdo dos laboratórios.

Referências

- Cloudflare (2024). What is a brute force attack? Acesso em: 10 maio 2026.
- ENISA — European Union Agency for Cybersecurity (2024). Enisa — about. Acesso em: 8 maio 2026.
- Gil, A. C. (2002). *Como elaborar projetos de pesquisa*. Atlas, São Paulo, 4 edition.
- IBM (2024a). What is a honeypot? Acesso em: 10 maio 2026.
- IBM (2024b). What is the CIA triad? Acesso em: 10 maio 2026.
- Kaspersky (2023). What is a honeypot? Acesso em: 12 maio 2026.
- Negi, P. S., Garg, A. K., and Lal, R. (2020). Intrusion detection and prevention using honeypot network for cloud security. In *Proceedings of the 10th International Conference on Cloud Computing, Data Science and Engineering (Confluence)*, pages 129–132, Noida, India. IEEE.
- NIST — National Institute of Standards and Technology (2018). Cyber ranges. Acesso em: 5 maio 2026.
- NIST — National Institute of Standards and Technology (2024). The NIST cybersecurity framework (CSF) 2.0. Acesso em: 5 maio 2026.

- OWASP Foundation (2021a). Cross site scripting (XSS). Acesso em: 1 maio 2026.
- OWASP Foundation (2021b). Insecure direct object reference prevention cheat sheet. Acesso em: 1 maio 2026.
- OWASP Foundation (2021c). OWASP top ten. Acesso em: 1 maio 2026.
- OWASP Foundation (2021d). Path traversal. Acesso em: 1 maio 2026.
- OWASP Foundation (2021e). SQL injection. Acesso em: 1 maio 2026.
- OWASP Foundation (2023). OWASP juice shop project. Acesso em: 20 maio 2026.
- PortSwigger (2024). Information disclosure vulnerabilities. Acesso em: 18 maio 2026.
- Saddiq, M., Broholm, M., and Pedersen, J. M. (2024). Cybersecurity training: Improving platforms through usability studies. In *Proceedings of the International Conference on Cyber Security and Resilience (Cyber Science 2023)*, pages 95–110, Singapore. Springer.
- Sikdar, S., Basu, M., and Kule, M. (2024). Honeypot deception: A clever approach of web intrusion detection and prevention. In *Proceedings of the International Conference on Communication Systems and Networks (COMSYS 2023)*, Singapore. Springer.
- Wood, R. (2023). DVWA — Damn Vulnerable Web Application. GitHub. Acesso em: 22 maio 2026.